Universität St.Gallen

Continual Learning | Class incremental

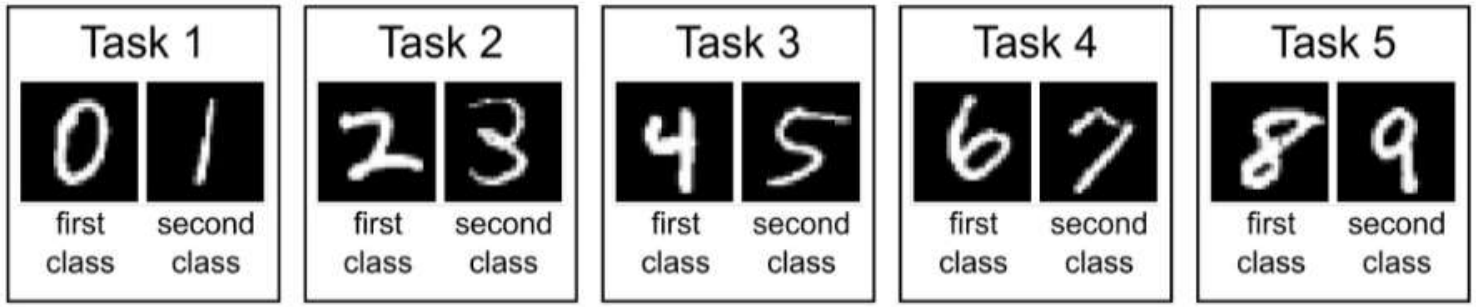
(2nd Coding lab– DL2025)

Continual learning Overview

AGI requires agents to be capable of learning and remembering many different tasks[1].
I.e. they must show a capacity of ***continual learning***.

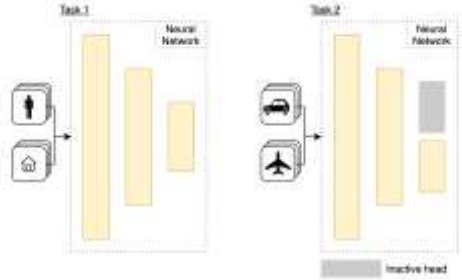
Challenge is the fact that as the model learn new tasks knowledge of previous tasks are lost -> ***catastrophic forgetting.***

The model needs enough **plasticity** to adapt to new knowledge but also **stability** enough to not forget previous knowledge.
This is known as the ***stability - plasticity dilemma.***



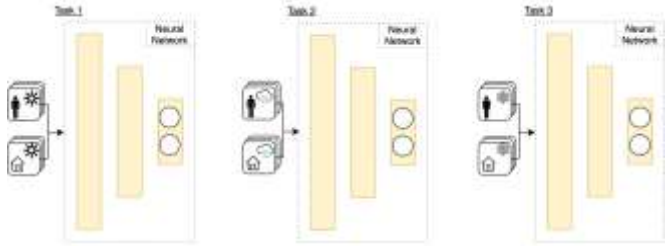
Continual learning scenarios

Task Incremental (TI)



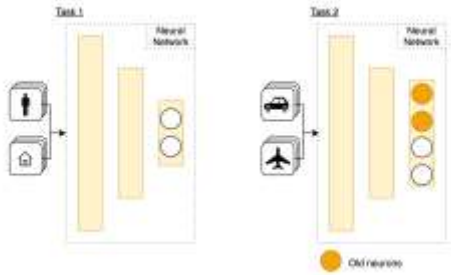
- Shared backbone but one head per task.
- Task identity is always provided.
- The easiest scenario.

Domain Incremental (DI)

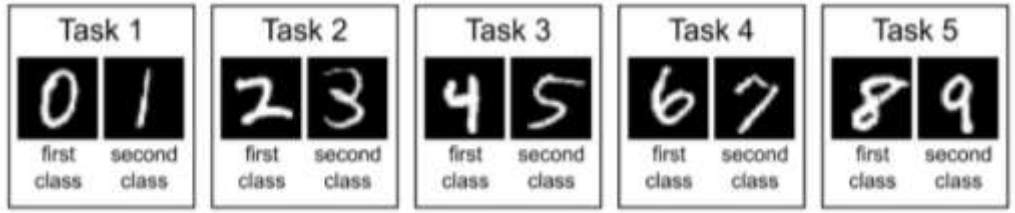


- Only solves the current task.
- Each task has the same classes.
- For example, images in different environments.

Class Incremental (CI)



- Each task introduces new classes that haven't been seen before.
- The hardest CL scenario.



SplitMNIST example

Task Incremental (TI)

With the task known is 1st or 2nd class, e.g. 0 or 1?

Domain Incremental (DI)

Is it a 1st or 2nd class, e.g. in [0,2,4,6,8] or in [1,3,5,7]?

Class Incremental (CI)

With task unknown which digit is it, with the choice being from 0 to 9?

Baselines and different strategies

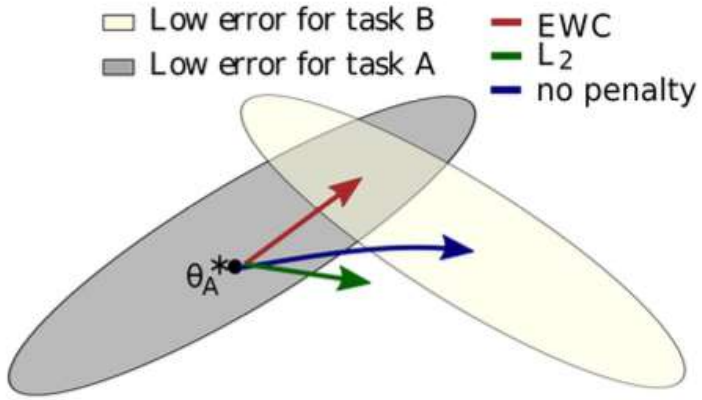
There are some baselines to compare different ways to improve continual learning.

- Naïve
  - The model is sequentially trained on all tasks as “normal” in a sequence, so just normal finetuning on each new task. This will cause forgetting and can be seen as the **lower bound**.
- Joint training
  - We take all the tasks and merge them together into one big dataset, this can be seen as the **upper bound**, as we don’t have anything to forget.

Elastic Weight Consolidation (EWC)

- Inspired by how we retain knowledge in our brain, "*task specific synaptic consolidation*". I.e. a **portion** of synapses are made more stable and thus retain information longer.

- EWC aims to replicate this behavior in neural networks by exploiting the overparameterization of DNNs, i.e. that many sets of weights will result in the same performance.



- So, if that is true there should be a solution for Task B that is close to the previous solution of Task A.

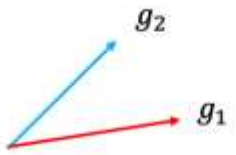
- The fisher information matrix[2] that will tell us how important a parameter is for a certain task and if its more important it will incur more penalty do change it. A quadratic penalty on the parameter is implemented.

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i(\theta_i - \theta^*_{A,i})^2$$

The function that we minimize in EWC

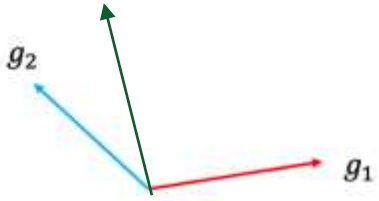
Gradient Episodic Memory (GEM)

- Episodic memory refers to memory of specific instances; in GEM we store a subset of previous examples.
- Instead of constraining the weights directly, as in EWC, GEM instead constrains the gradient. I.e. how parameters are allowed to move.
- GEM compares the current tasks gradient with gradients computed on old memory samples. If the gradients are aligned all is good but if the new gradient would *increase* the loss on past tasks, it is **projected** onto the **closest direction** that leaves past losses unchanged or improved.
- In the very simple example below g_2 is simply projected to the orthogonal direction from g_1. But this is more complex when we then get more tasks.



$g_1 \cdot g_2 \geq 0$

The gradients are aligned
+ Positive transfer



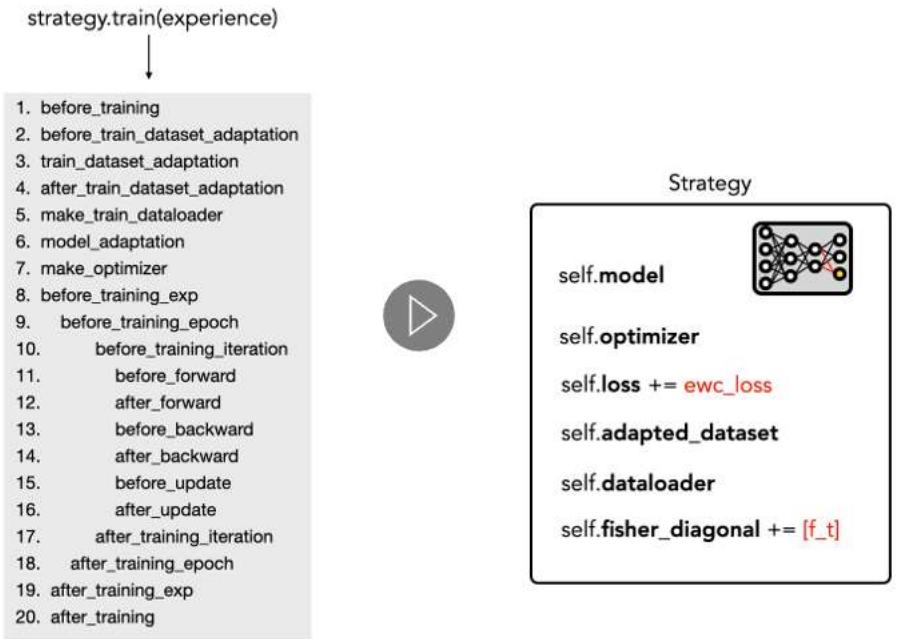
$g_1 \cdot g_2 \leq 0$

The gradients are not aligned
- Negative transfer

Templates in Avalanche

As an *avalanche* continual learning incrementally improves the learning

- The strategies in Avalanche are built on templates.
- Each phase of training is split into a bunch of points where you can insert a plugin that can modify the behavior.
- All strategies follow the same flow with the differences often just being the plugins at certain points.



The end